

Scaling APIs Like a City That Never Sleeps

Building Resilient APIs in Azure

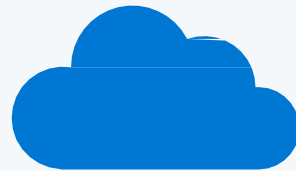


Agenda

Building Resilient and Scalable APIs in Azure



Intro & Problem Space



**Azure Toolset
Overview**



**High Availability &
Disaster Recovery**



Resilience Strategies



**Monitoring &
Observability**



Live Demo



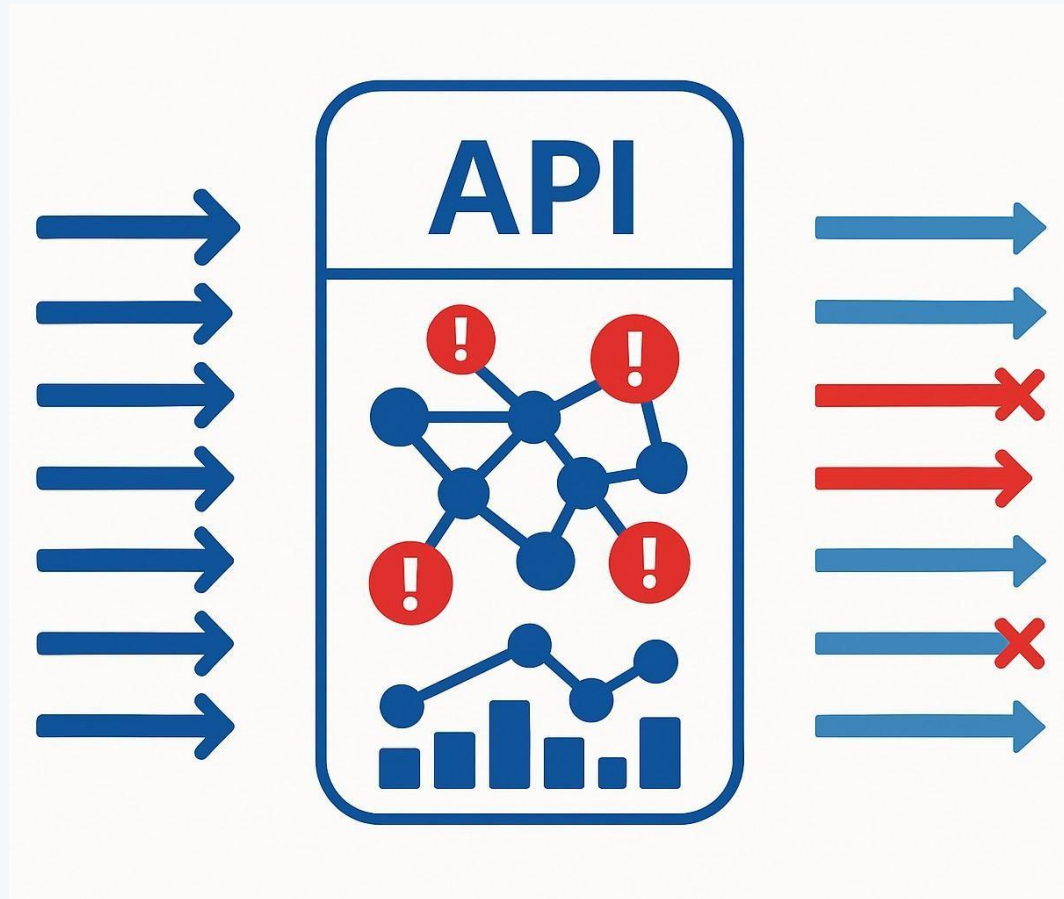
Blueprint & Wrap Up



Q&A

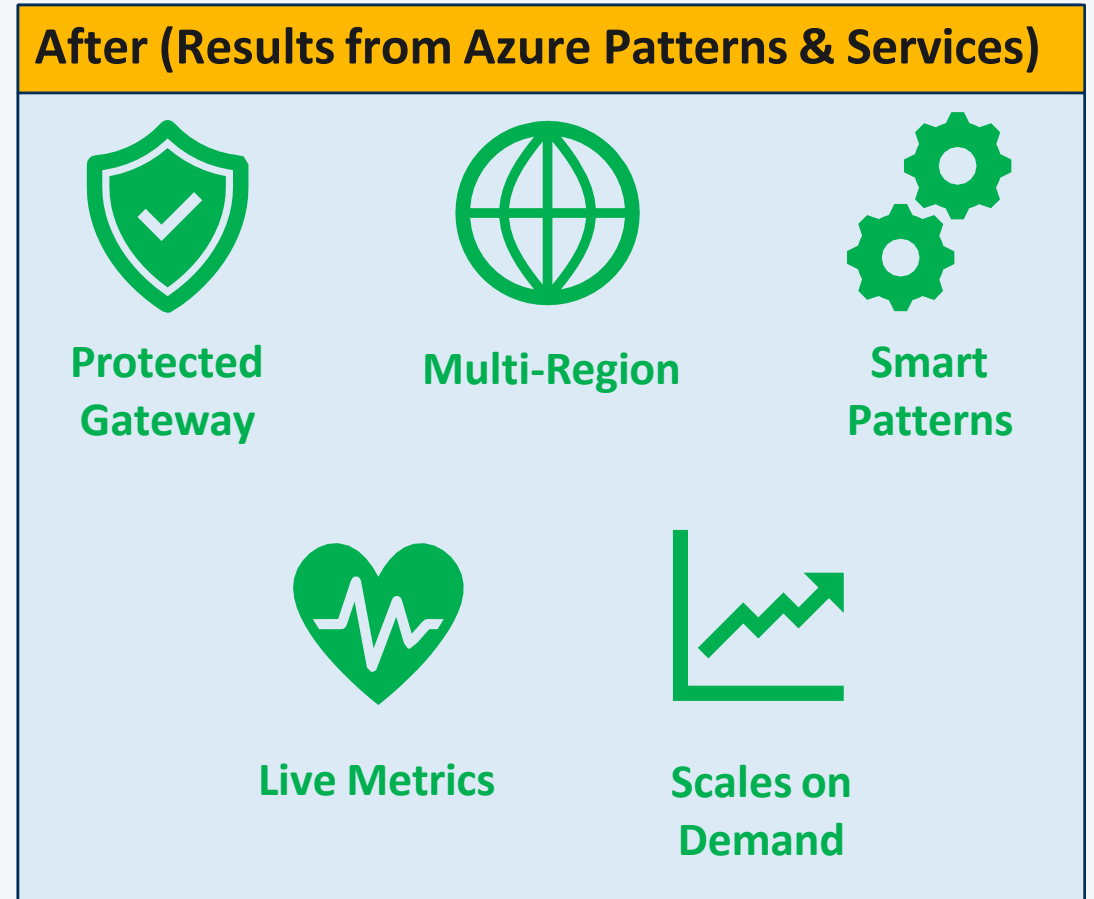
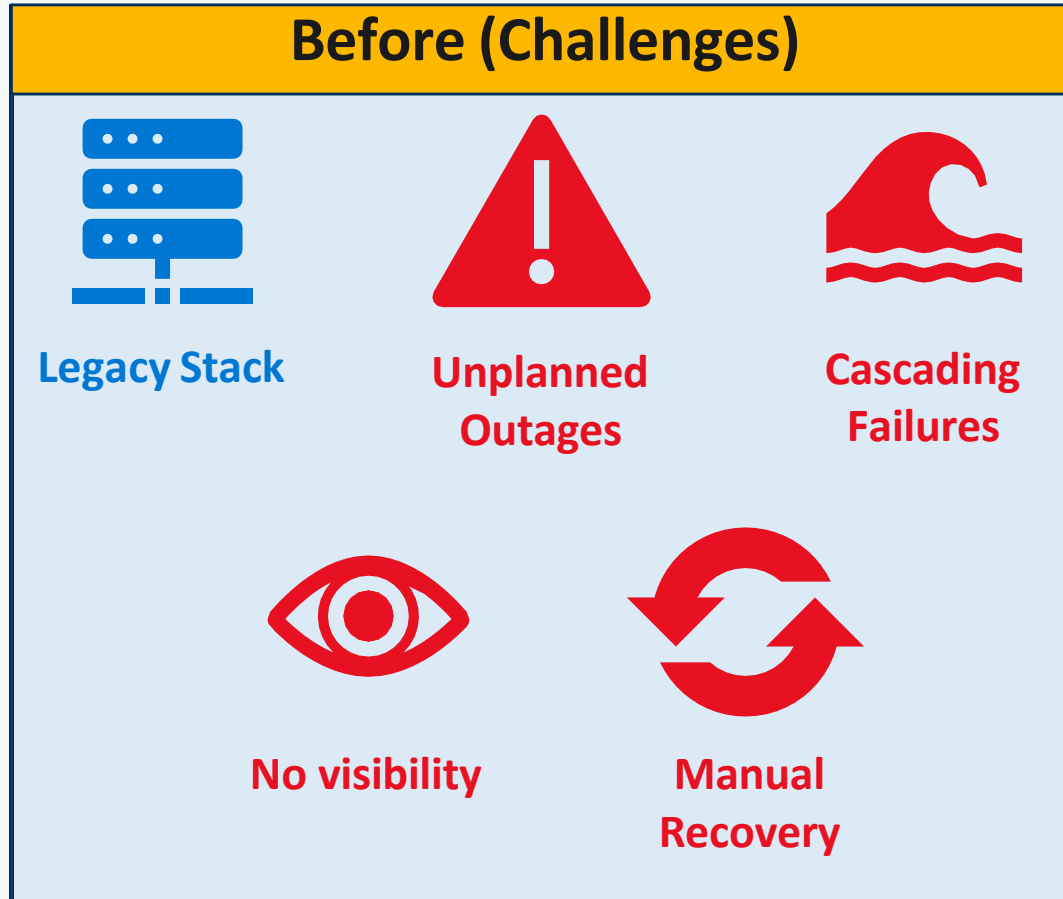
The Resilience Challenge

Building Resilient and Scalable APIs in Azure



Resilience Turnaround

From Fragile to Fault-Tolerant: An API Resilience Transformation



Azure Toolset Overview

Building Resilient APIs in Azure



Azure Toolset Overview

Primary Trio



**Azure API
Management**

**Traffic Shaping
and Throttling**

Caching

**Policy
Enforcement**

**Backend
Shielding**

Building Resilient and Scalable APIs in Azure

Azure Toolset Overview

Primary Trio



Azure API
Management



Azure Functions

Elastic Capacity

**Scale-to-Zero
Efficiency**

**Event-Driven
Triggers**

**Isolation of
Workloads**

Azure Toolset Overview

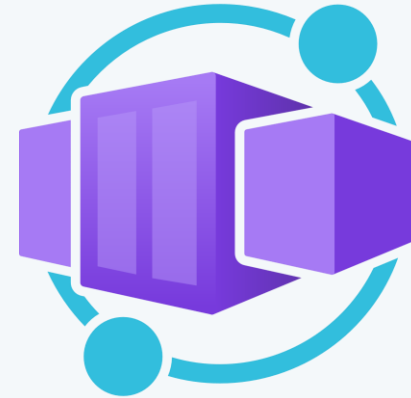
Primary Trio



Azure API
Management



Azure Functions



**Azure Container
Apps**

Granular Scaling

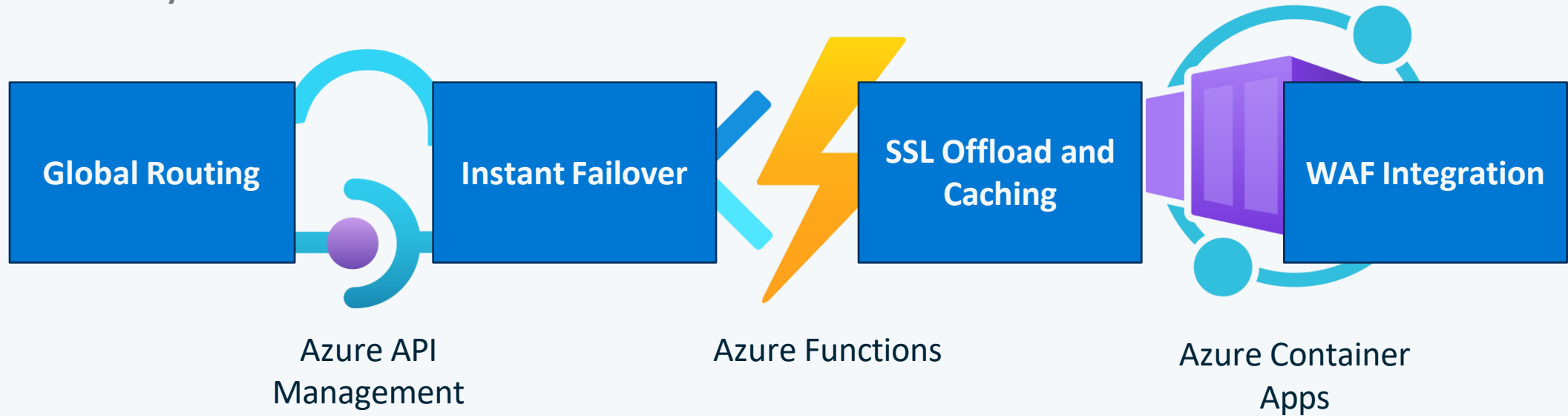
**Multiple Scale
Triggers**

**Revisions and
Blue-Green
Deployments**

**Service-to-
Service
Communication**

Azure Toolset Overview

Primary Trio

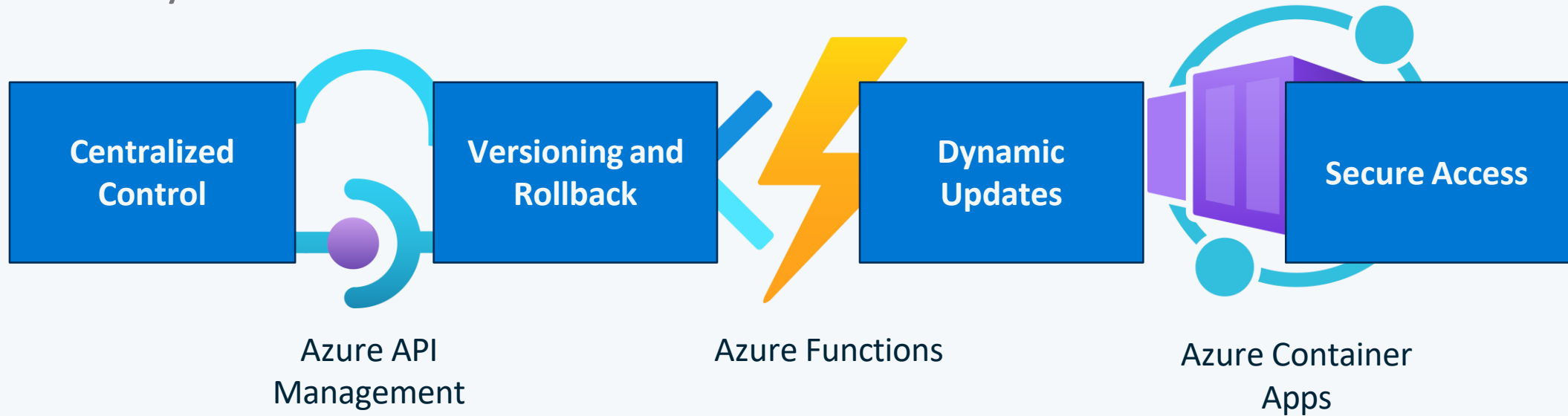


Azure
Front Door

Building Resilient and Scalable APIs in Azure

Azure Toolset Overview

Primary Trio



Azure
Front Door



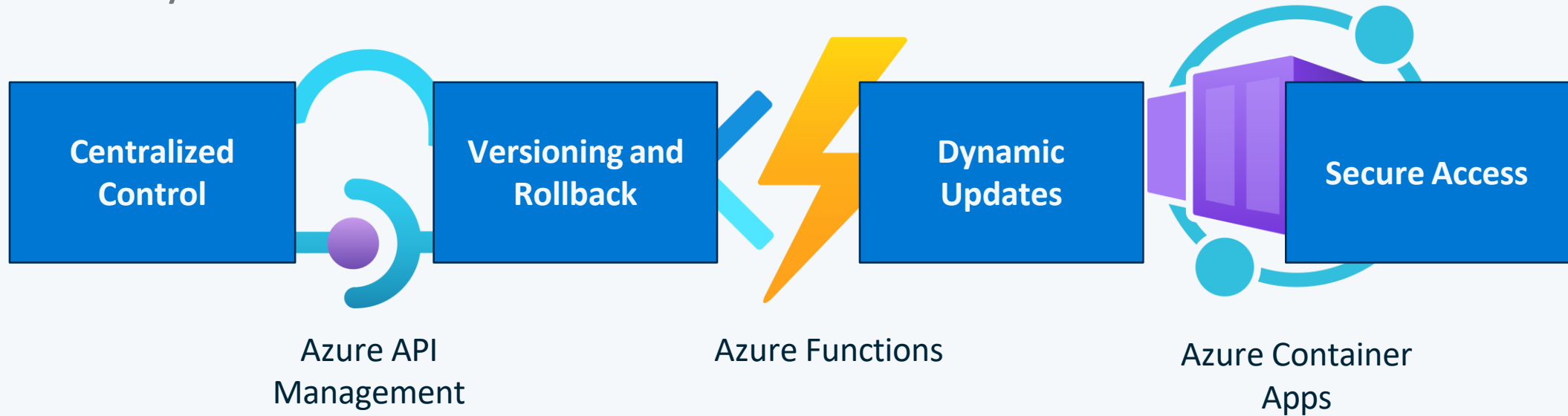
Azure
Monitor



Application
Insights

Azure Toolset Overview

Primary Trio



Azure
Front Door



Azure
Monitor



Application
Insights



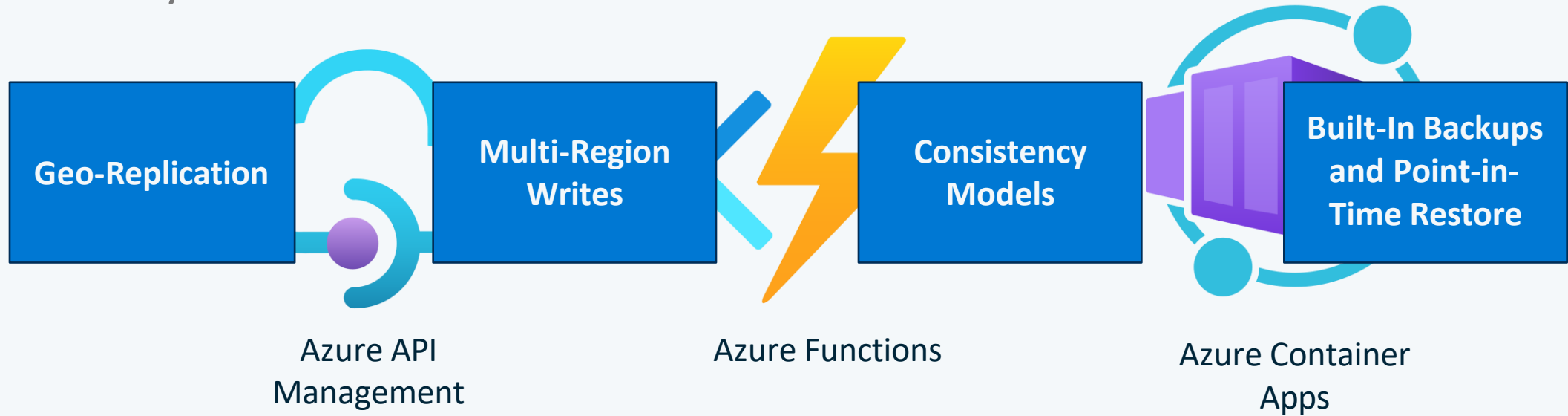
Azure Key
Vault



Azure App
Configuration

Azure Toolset Overview

Primary Trio



Azure
Front Door



Azure
Monitor



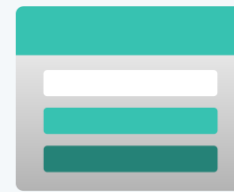
Application
Insights



Azure Key
Vault



Azure App
Configuration



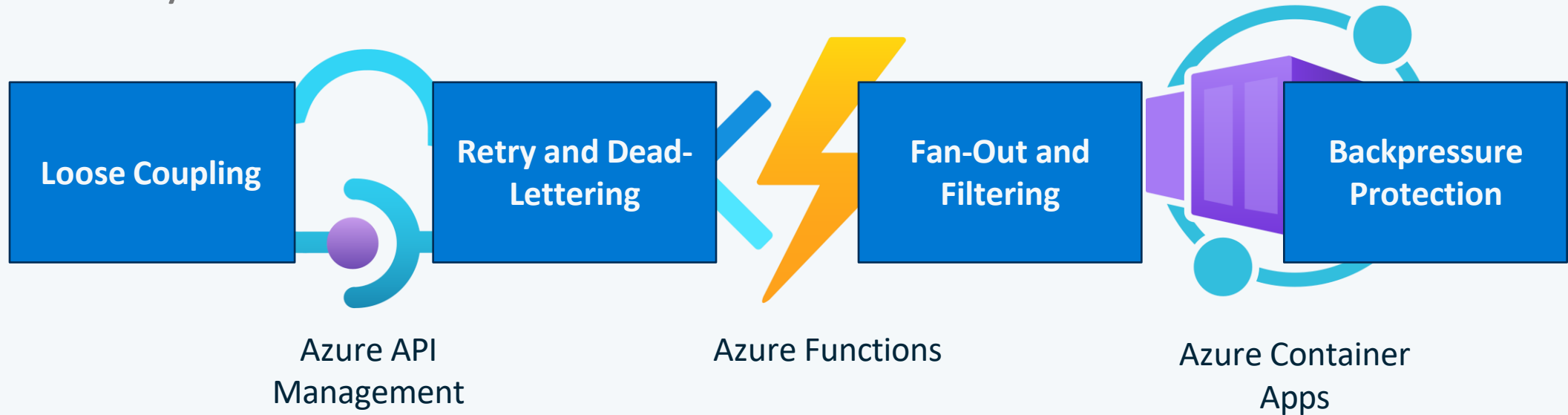
Azure
Storage



Cosmos
DB

Azure Toolset Overview

Primary Trio



Azure
Front Door



Azure
Monitor



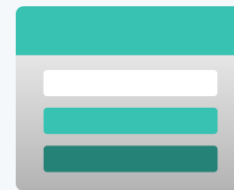
Application
Insights



Azure Key
Vault



Azure App
Configuration



Azure
Storage



Cosmos
DB

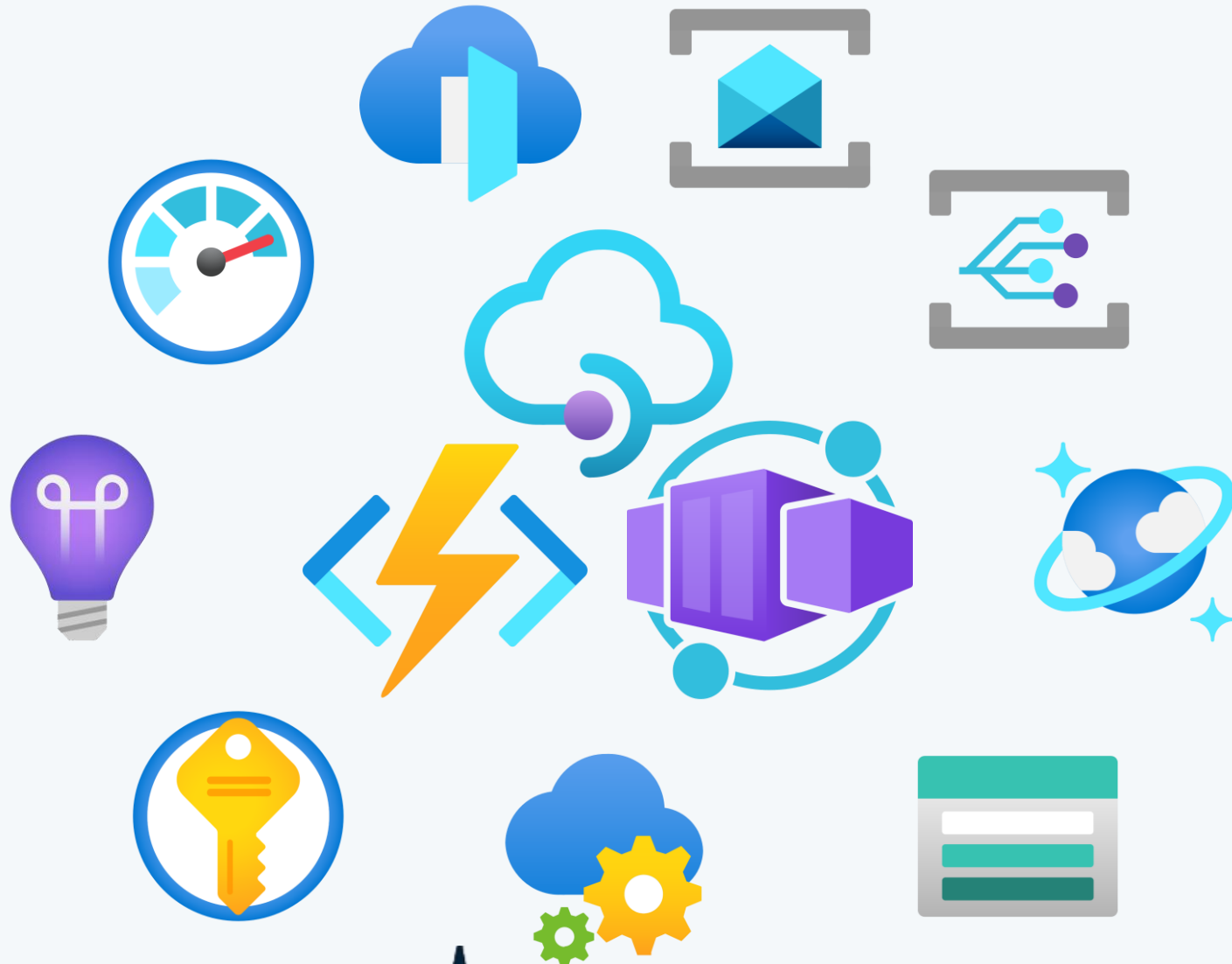


Event Grid



Service
Bus

Azure Toolset Overview



High Availability & Disaster Recovery Design

Building Resilient APIs in Azure



Framing the Discipline

High Availability vs. Disaster Recovery

High Availability (HA)
Keep Running

Disaster Recovery (DR)
Recover When Stopped

Recovery Time Objective (RTO)

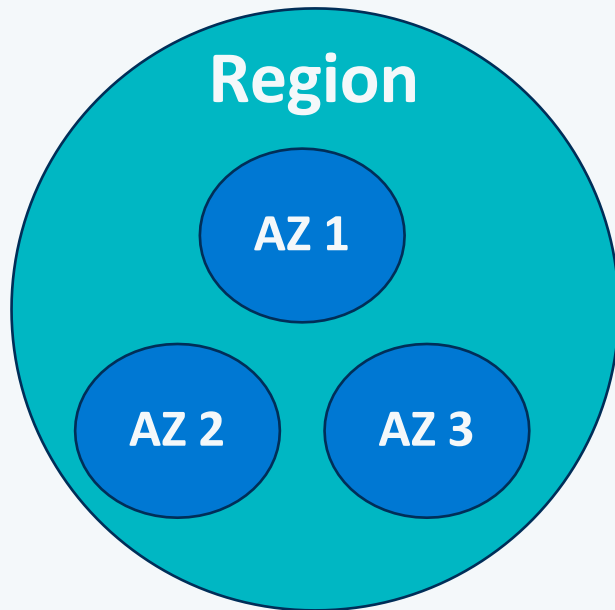
Recovery Point Objective (RPO)

Designing for Redundancy

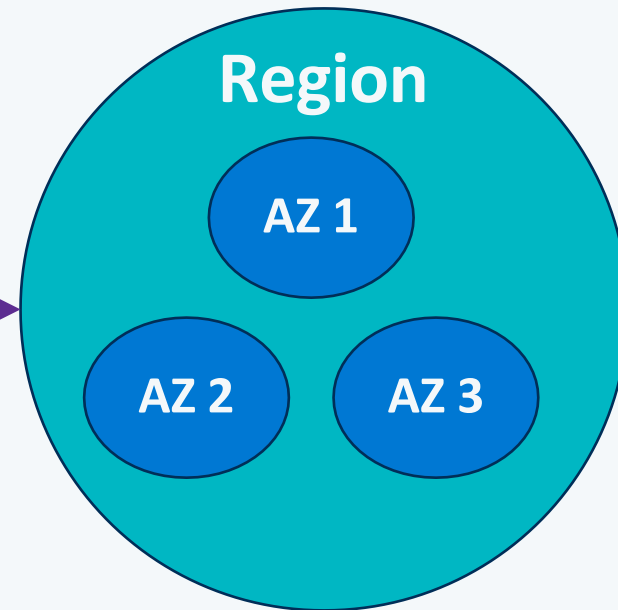
Availability Zones & Region Pairs

Geographic Redundancy

Availability Zones



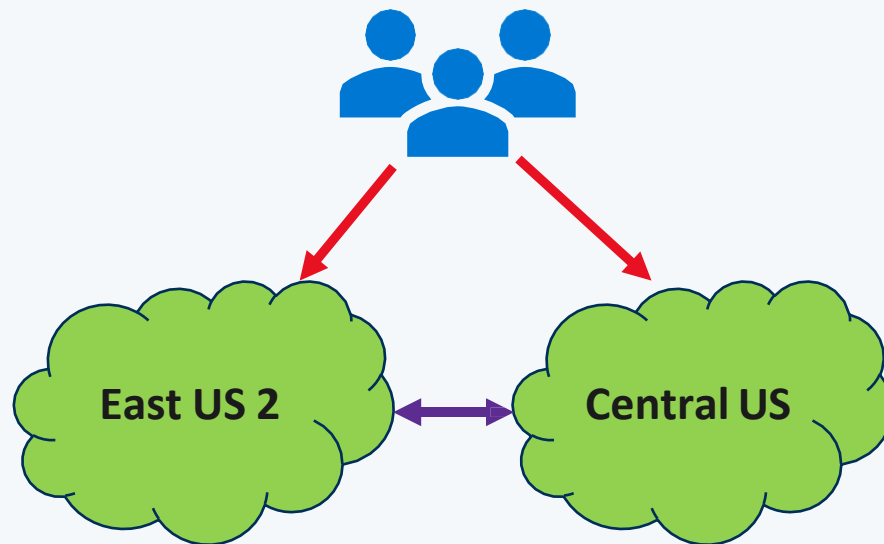
Region Pairs



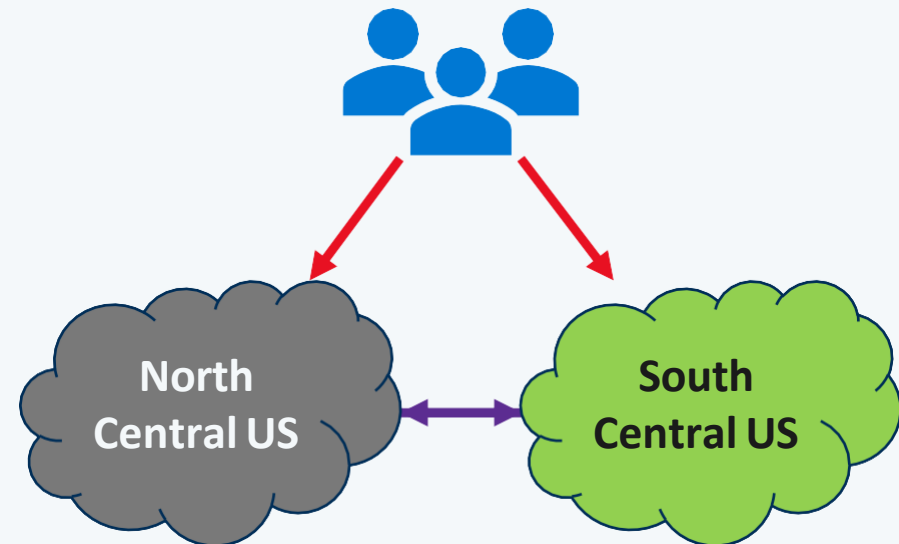
Deployment Strategies for Resilience

Active-Active vs Active-Passive

Active-Active

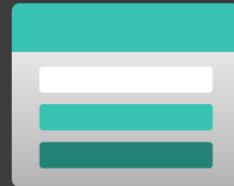
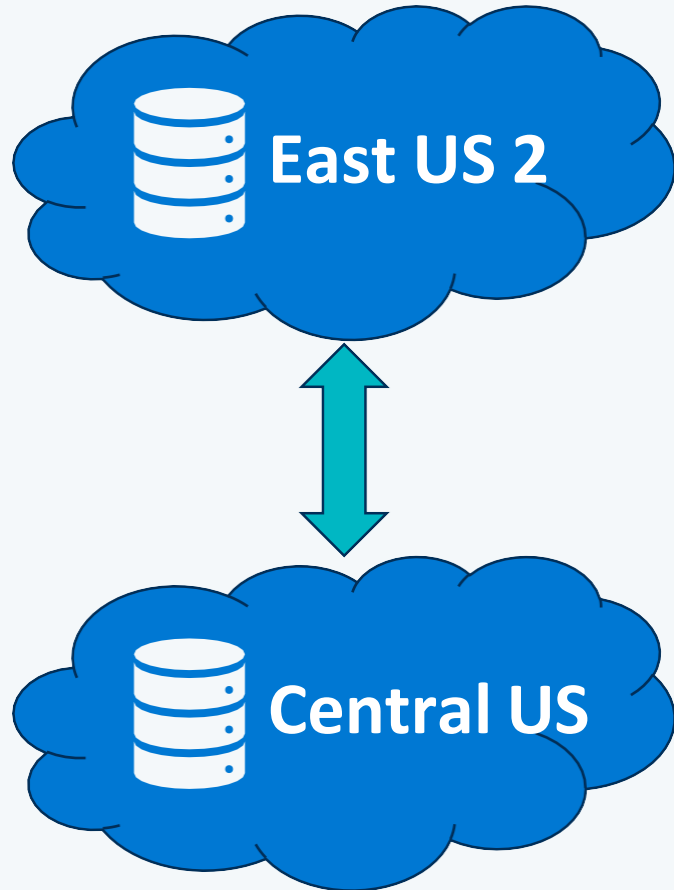


Active-Passive



Keeping Data Durable & Available

Data Resilience & Replication



Azure
Storage

- Geo-redundant (GRS, RA-GRS)
- Automatic replication across regions
- Blob, file, queue, table storage



Cosmos DB

- Multi-region writes, active-active
- Tunable consistency models
- Automatic failover groups

Operational Readiness

Practice Resilience

Key Practices

- Monitoring & Alerts
- Chaos Testing / Game Days
- Runbooks & Automation

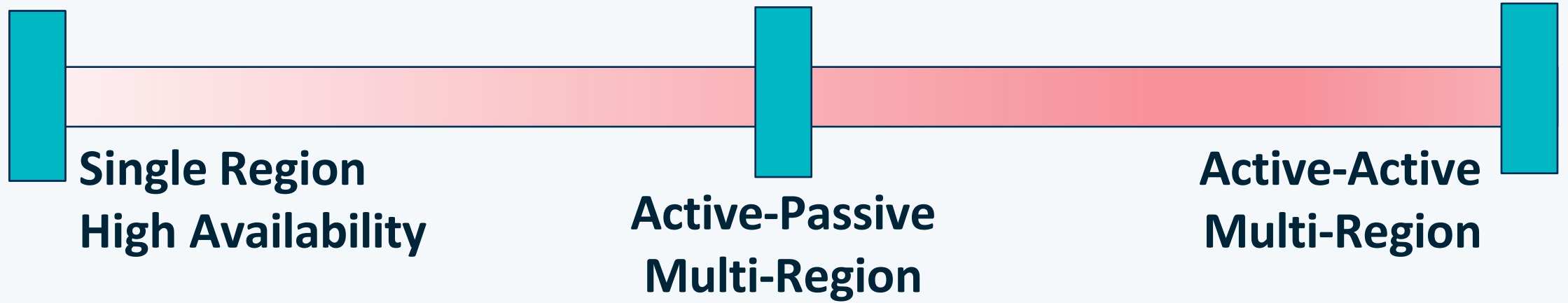
Outcomes

- Faster detection of failures
- Validated assumptions
- Predictable, repeatable recovery



Trade-Offs & Cost Lens

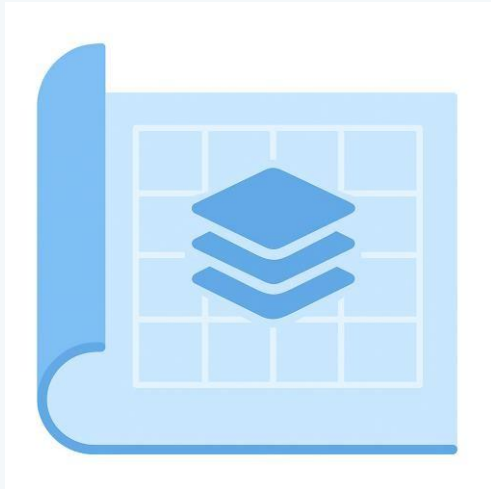
Balancing Cost, Complexity, and Risk



Choose the level of resilience that matches your business risk.

From Principles to Practice

Recap & Transition



- High Availability = *survive everyday failures*
- Disaster Recovery = *recovery from rare, catastrophic events*
- Tools: Zones, Region Pairs, Active-Active/Passive, Data replication, Ops Readiness, Cost Trade-offs

Resilience Strategies

Building Resilient APIs in Azure



Retry Policies

Smarter Recovery from Failures



Concepts

- Exponential Backoff – wait longer between retries
- Jitter – add randomness to avoid retry storms
- Implementation – SDKs, Polly, APIM retry policies

Why It Matters

- Prevents overwhelming a struggling service
- Improves recovery time during outages
- Reduces cascading failures

Circuit Breakers

Failing Fast to Protect the System



Concepts

- Detect repeated failures
- Block further requests while the service is unhealthy
- Allow recovery attempts after cooldown

Why It Matters

- Prevents cascading failures
- Stops clients from overwhelming a failing service
- Enables faster recovery once health improves

Throttling & Rate Limiting

Protecting Capacity



Concepts

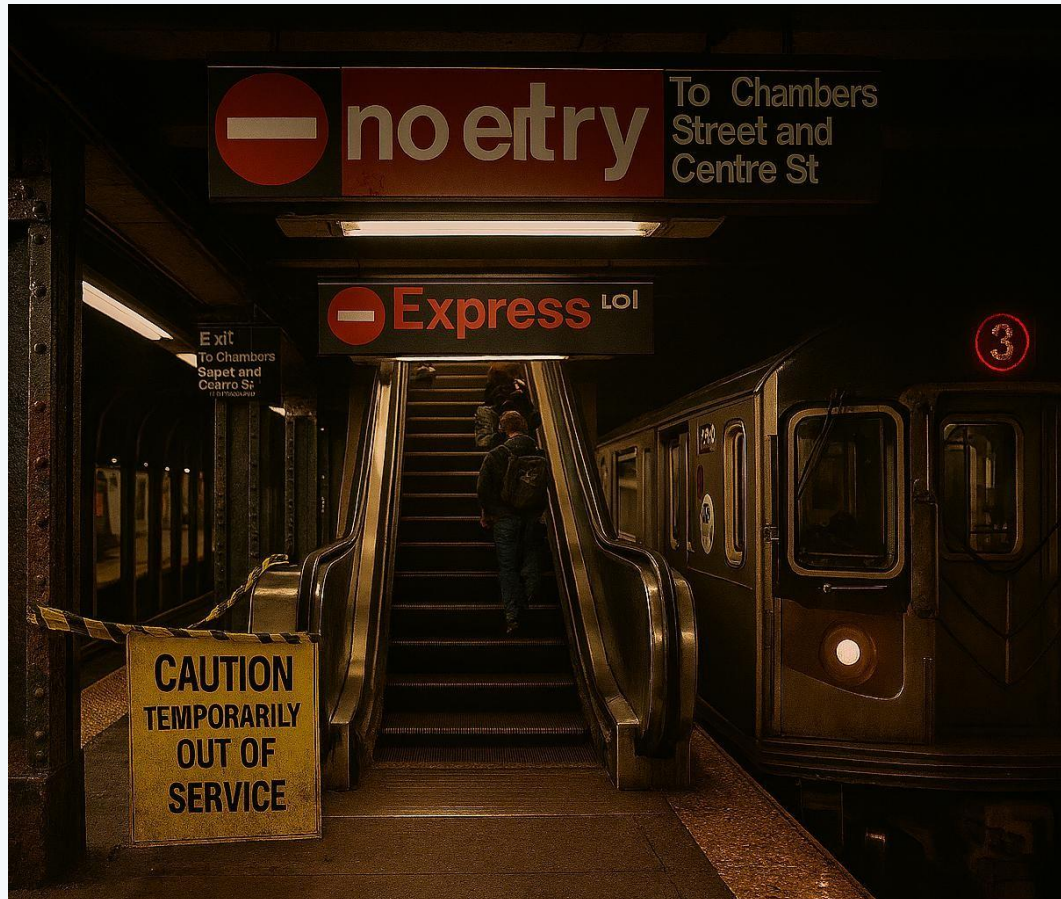
- **Rate Limiting:** cap requests per second/minute
- **Throttling:** slow down or reject excess requests

Why It Matters

- Prevents overload and cascading failures
- Ensures fair usage across clients
- Preserves critical functionality under stress

Graceful Degradation

Preserving the Core



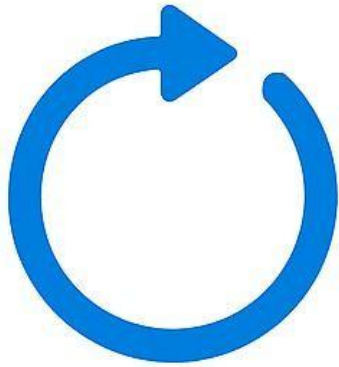
Concepts

- Disable or reduce non-essential features
- Serve cached or static data when live data is unavailable
- Switch to read-only-mode for critical services
- Provide clear use messaging

Why It Matters

- Keeps the core business function alive
- Minimizes user frustration and operational damage
- Buys time for recovery without total outage

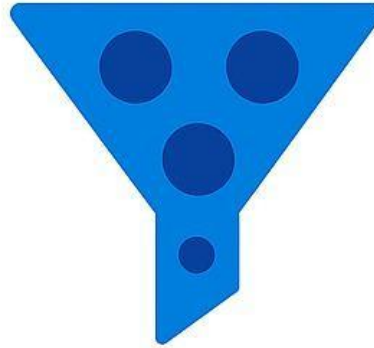
Recap



Retry Policies



Circuit Breakers



**Throttling
and Rate
Limiting**



Graceful Degradation

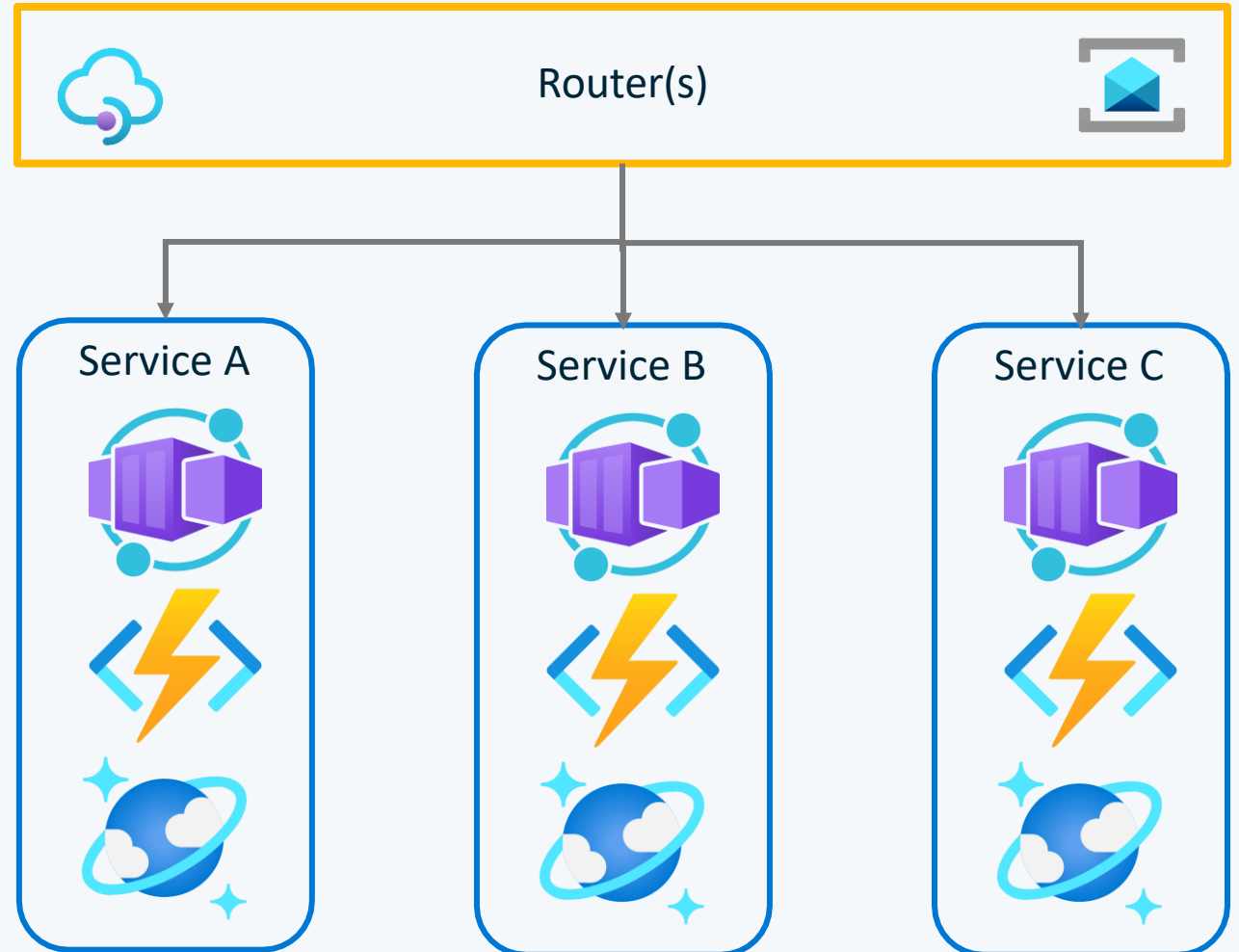
Architecture Patterns

Building Resilient APIs in Azure



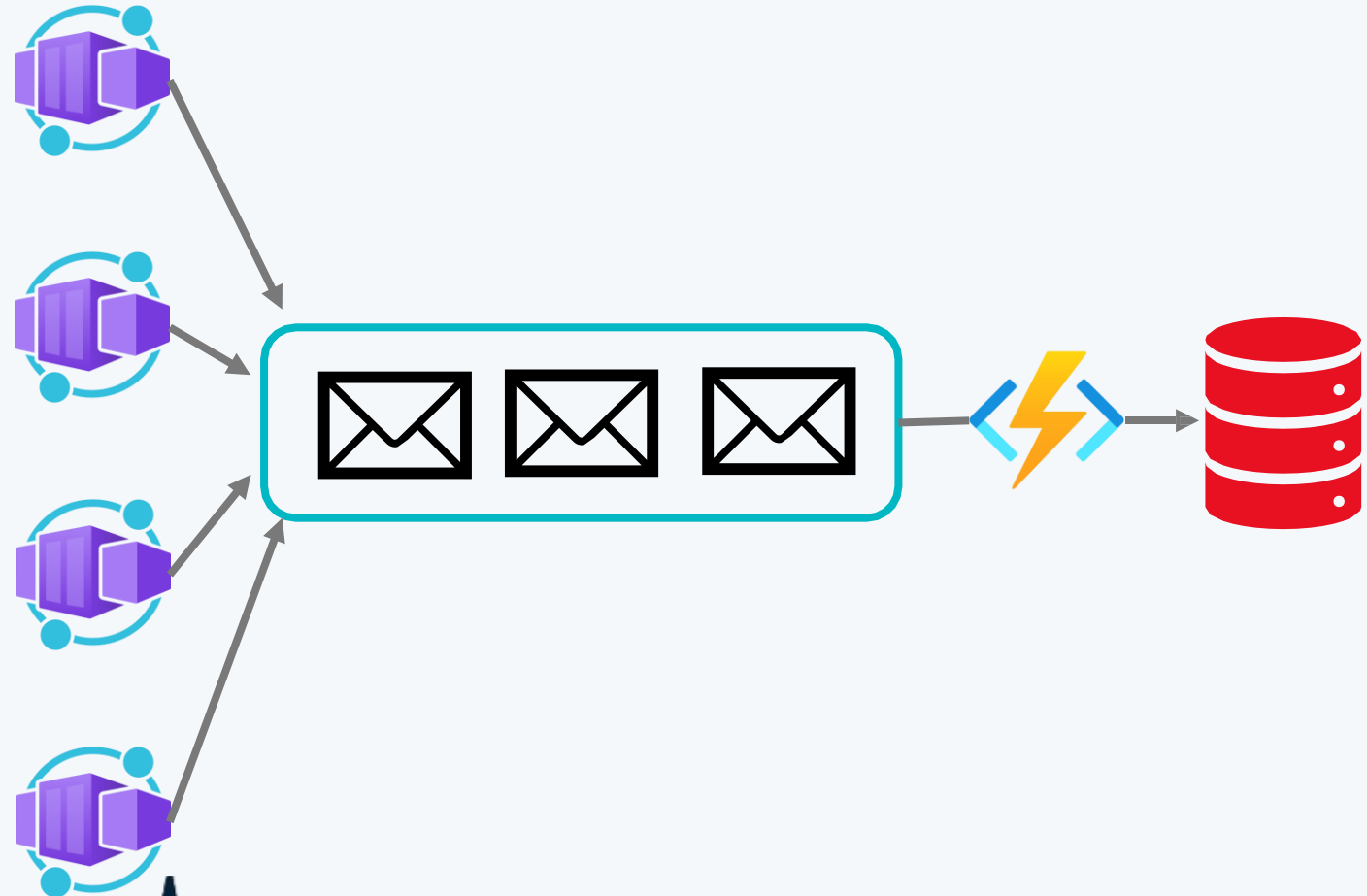
Bulkhead Isolation

Partitioning for Resilience



Queue-Based Load Leveling

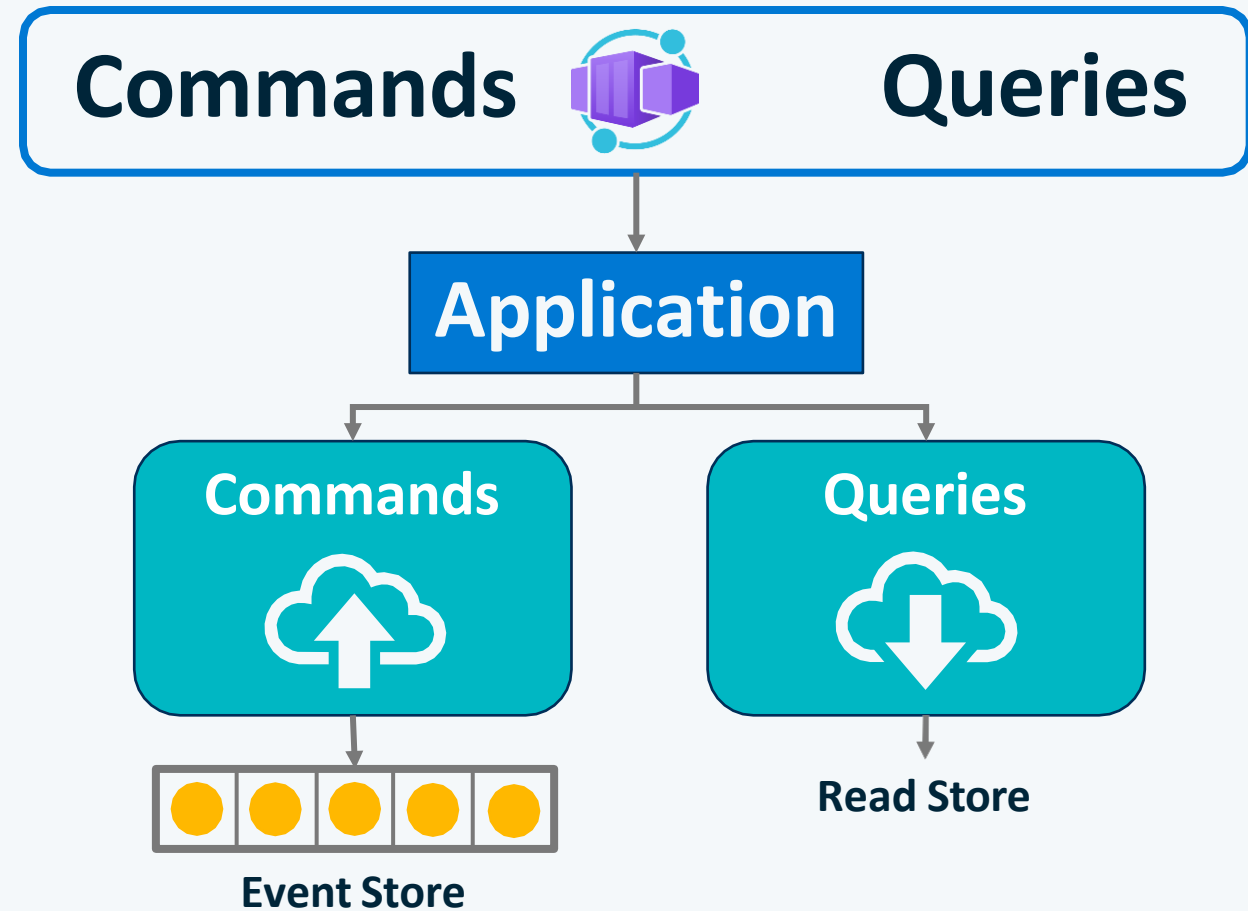
Buffering for Stability



Recording for Recovery

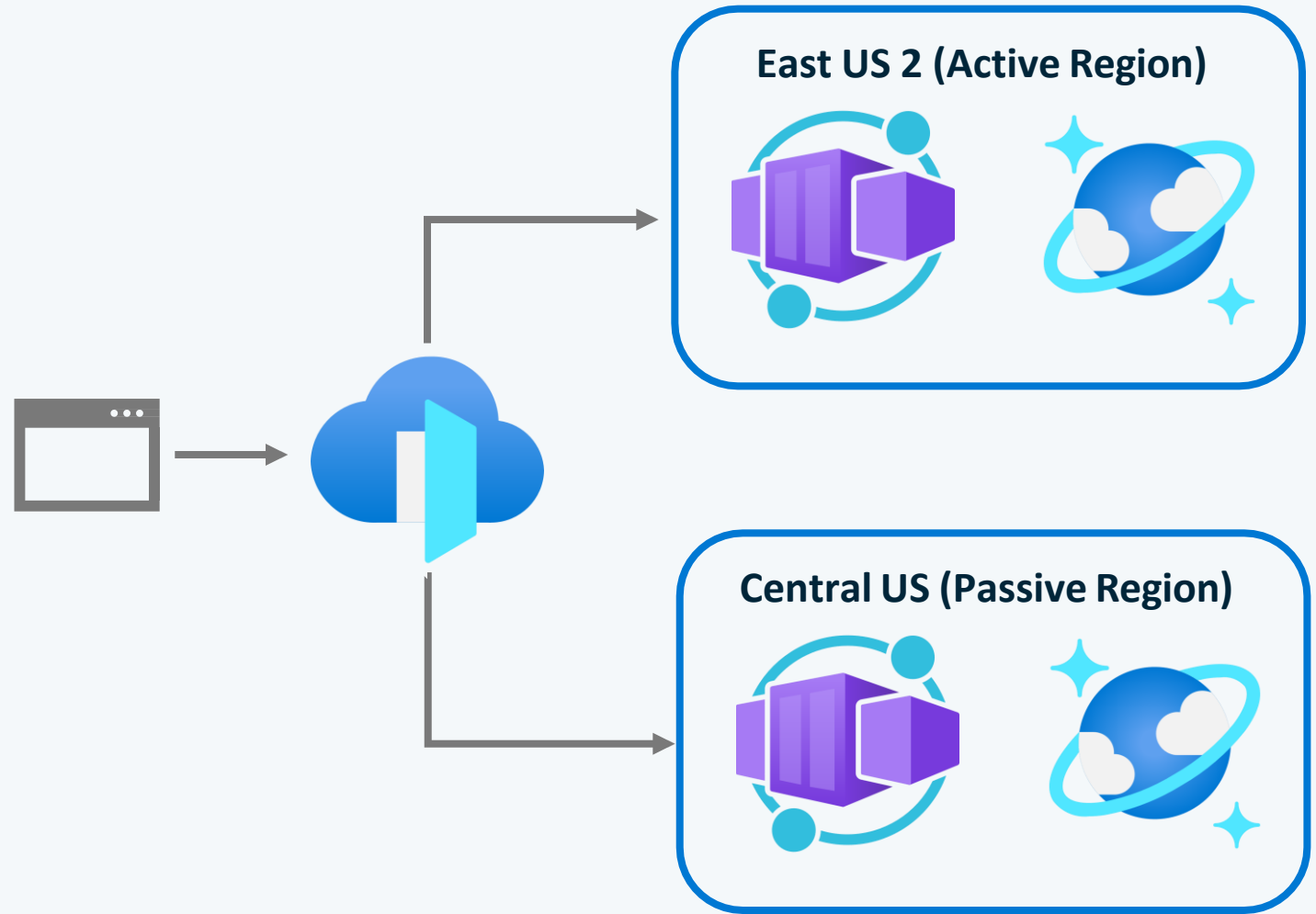
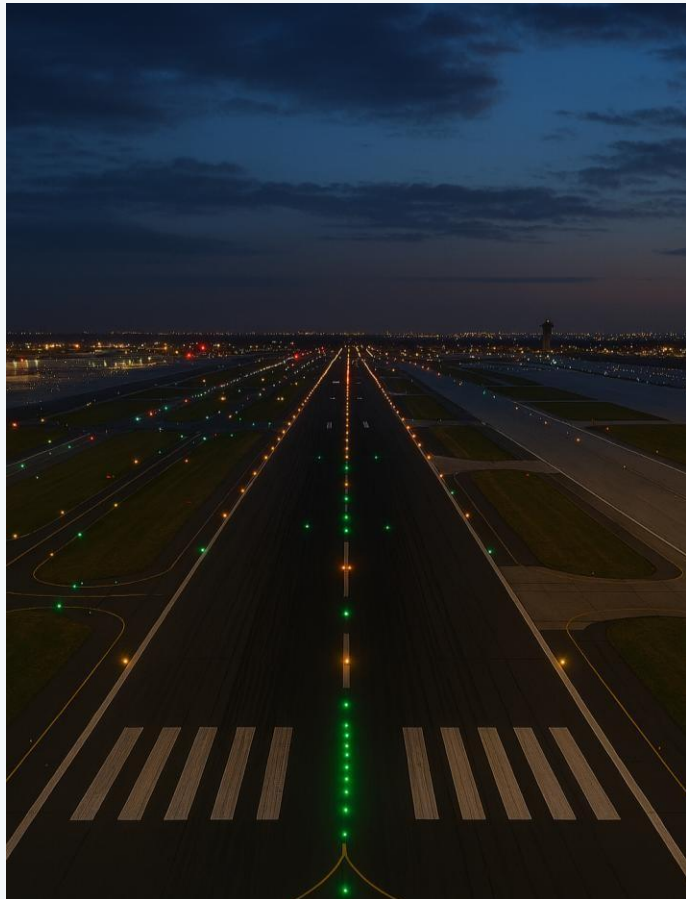
NEW YORK SUBWAY STATION PLATFORM ACTIVITY LOG

STATION: _____ DATE: _____ AGENT _____

[illegible]

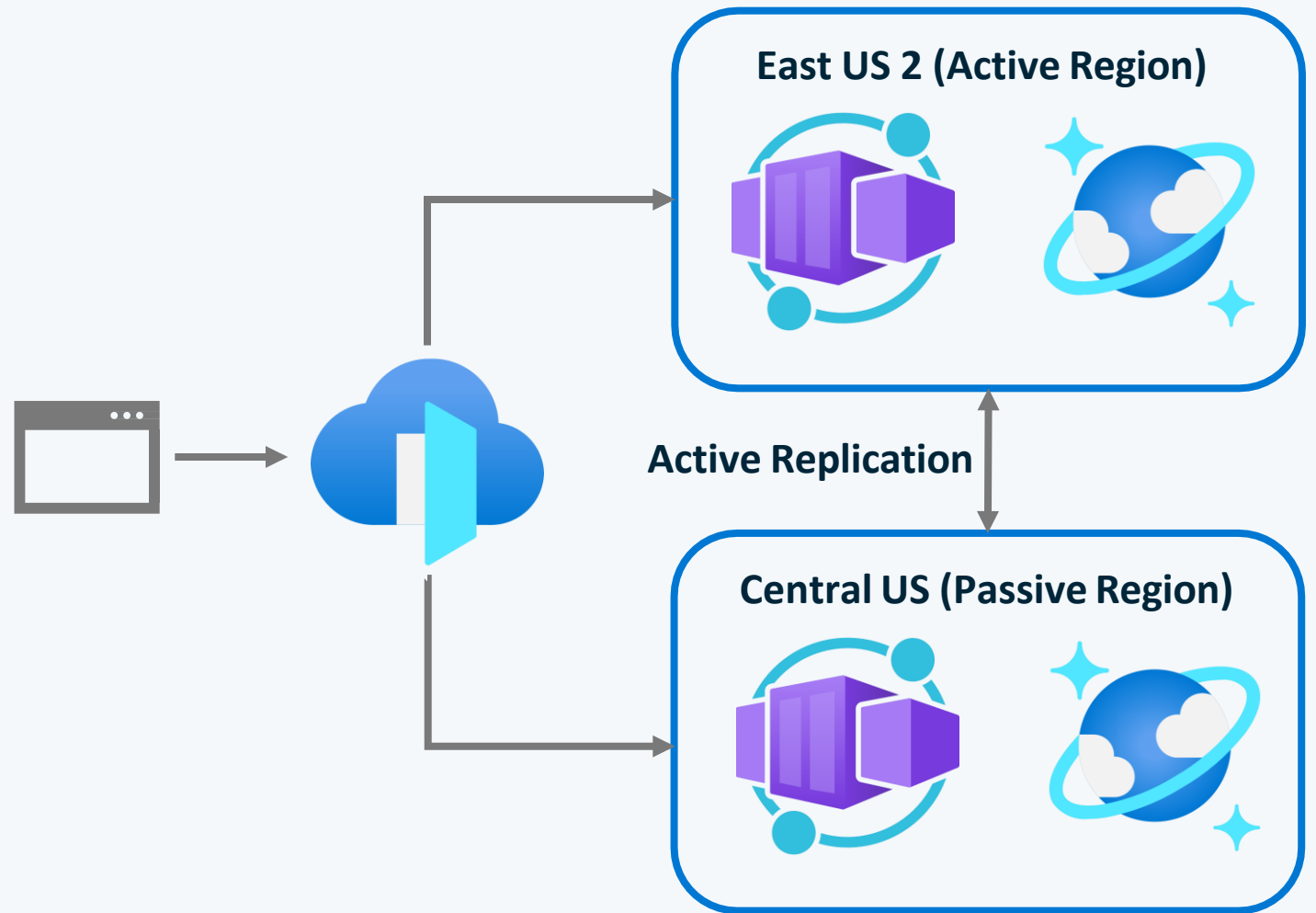
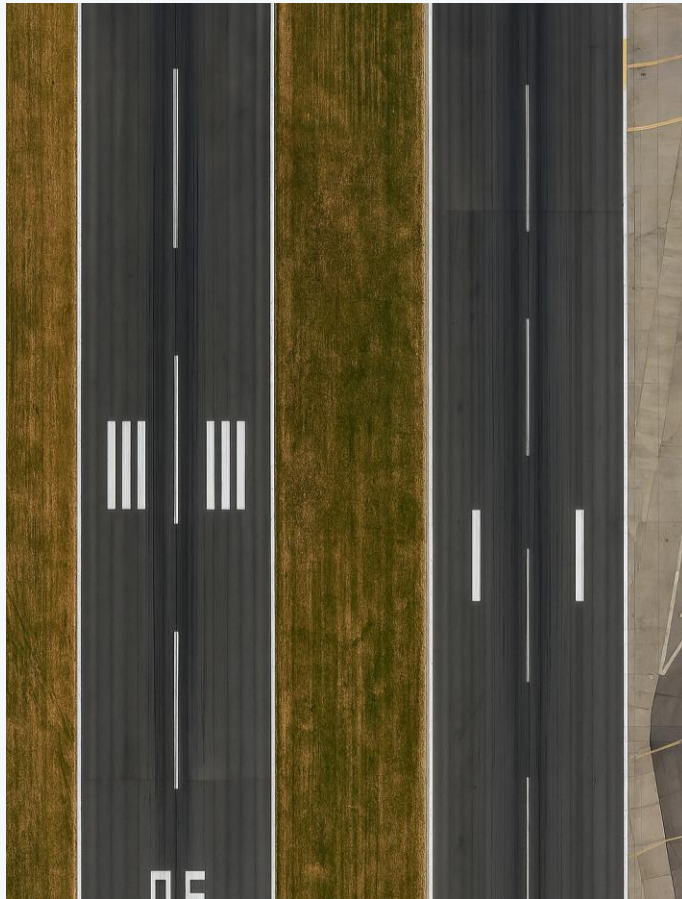
Multi-Region Active/Passive

Replicating for Redundancy



Multi-Region Active/Active

Replicating for Redundancy



Recap

Architecture Patterns



**Bulkhead
Isolation**



**Queue-Based
Load Leveling**



**Event Sourcing
& CQRS**



**Multi-Region
Active/Passive
& Active/Active**

Monitoring & Observability

Building Resilient APIs in Azure



The Tools of Observability



The Tools of Observability



Monitor



Application Insights



**Log Analytics
Workspaces**

The Tools of Observability



Monitor



Application Insights



**Log Analytics
Workspaces**



Metrics



Network Watcher



Diagnostic Settings



Workbooks



Activity Log

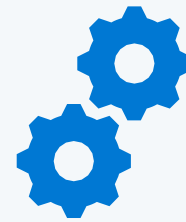
The Observability Loop



Detect



Decide

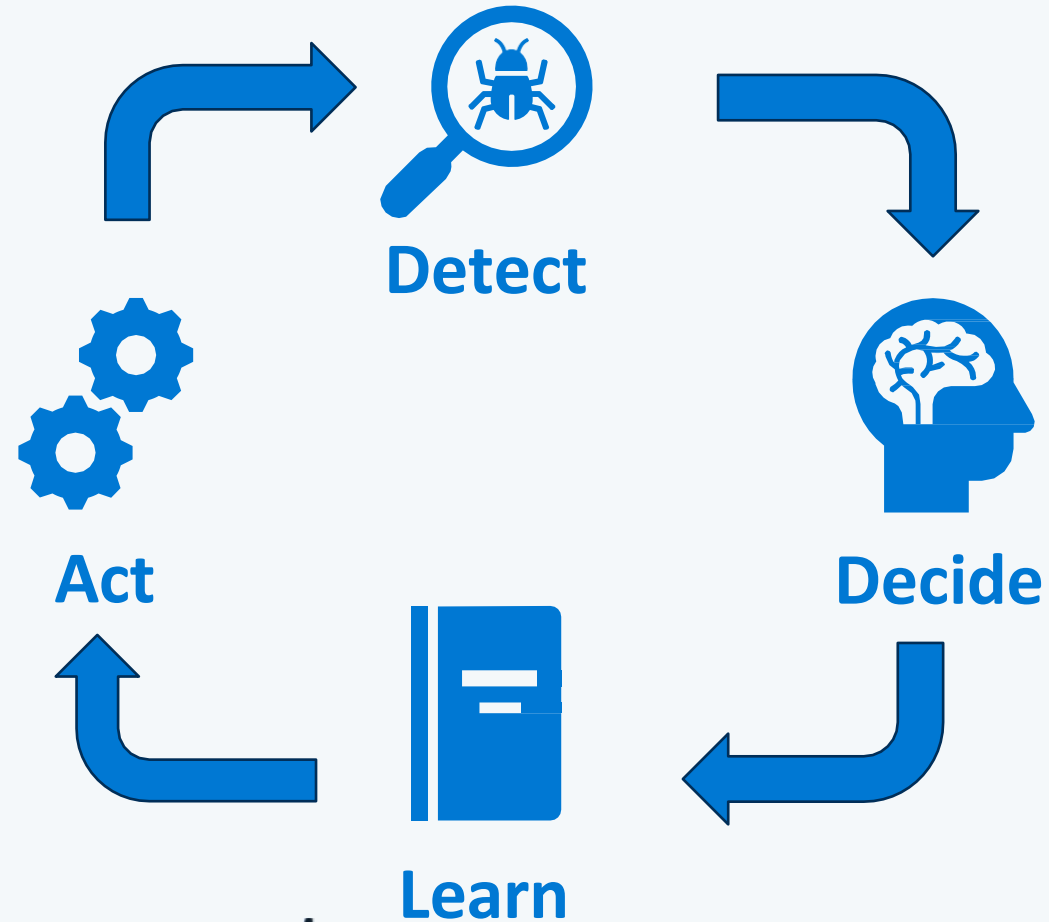


Act

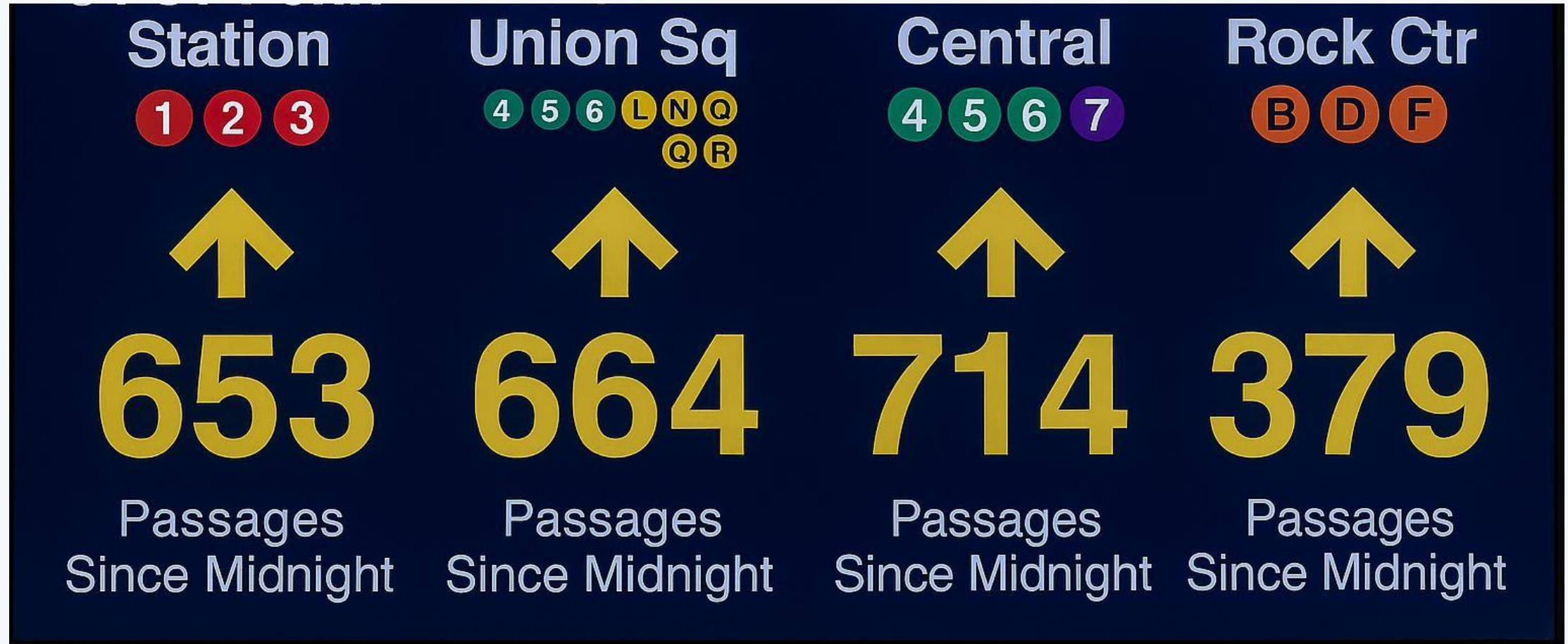


Learn

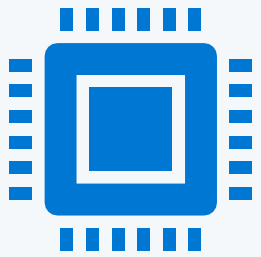
The Observability Loop



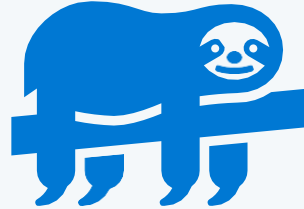
From Signals to Scaling



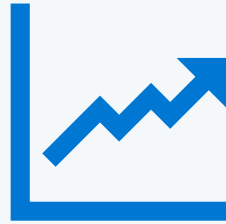
From Signals to Scaling



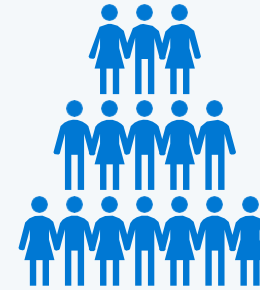
CPU /
Memory



Request
Latency



Error Rate



Queue Depth



Dependency
Failures

Keeping the System on Track



Observability

Design Blueprint

Building Resilient APIs in Azure

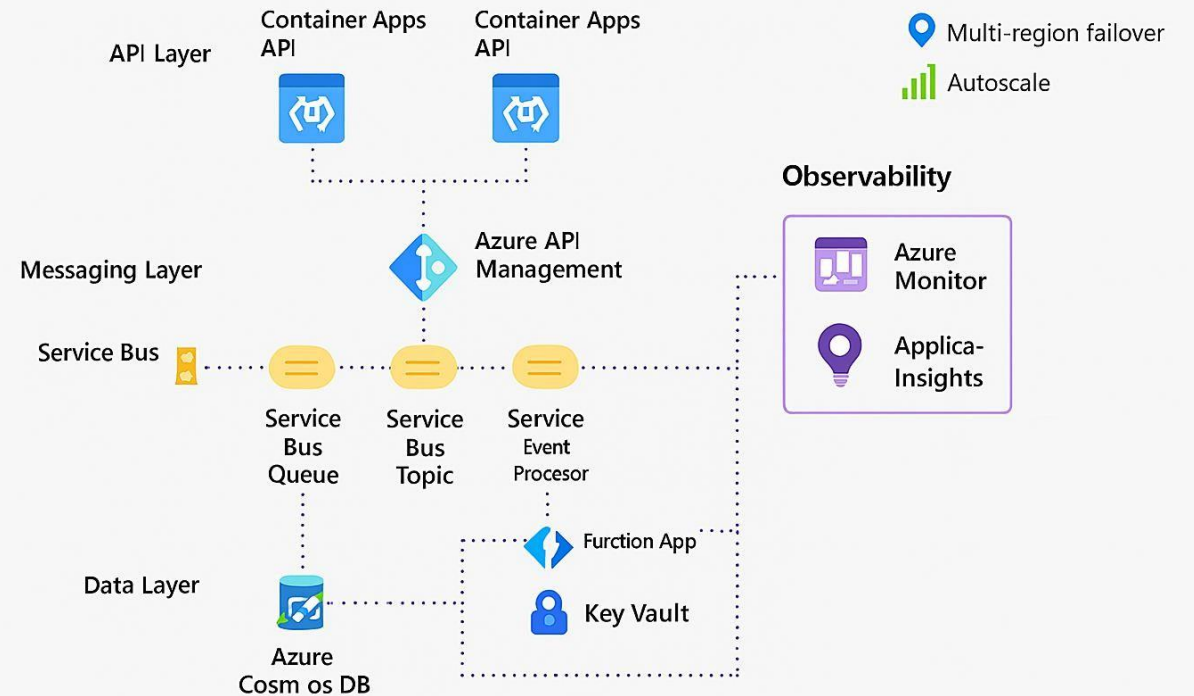


Putting It All Together

Design Blueprint

- API Layer (Top)
- Messaging Layer (Middle)
- Data Layer (Bottom)
- Observability (Side Panel)
- Resilience Features (Top Right)

Design Blueprint: Putting It All Together



Checklist for Resilient APIs

- Implement multi-region failover
- Use autocal rules for API workloads
- Buffer traffic with Service Bus queues
- Broadcast events with Service Bus topics
- Monitor latency, error rates, and queue depth
- Trigger alerts and scale actions from metrics
- Trace requests end-to-end with App Insights
- Secure secrets with Key Vault
- Design for observability day one

Q&A

- Intro & Problem Space
- Azure Toolset Overview
- High Availability & Disaster Recovery
- Resilience Strategies
- Monitoring & Observability
- Blueprint & Wrap Up
- Q&A



Slides & Reach Out

✉ chadgreen@chadgreen.com

💬 TaleLearnCode

🌐 ChadGreen.com

🐦 ChadGreen & TaleLearnCode

📘 ChadwickEGreen

